```
    public String finalStatus() {
        finalStatus = ...
        return finalStatus;
    }
}
```

Invoke the production code to determine the final status value.

Concordion uses a similar approach, except that the requirements are expressed in the form of simple HTML pages rather than on a wiki.

Both these approaches give you a great deal of flexibility in how you build your living documentation. But this flexibility does come at a cost—you need to organize and maintain the documentation pages, and the links between the pages, by hand, and it's much harder to do the sort of high-level aggregation reporting you've seen in the previous sections.

11.6 Technical living documentation

Living documentation doesn't stop with the high-level requirements. An important part of the low-level BDD practices you saw in chapter 10 involves writing unit and integration tests in the form of low-level executable specifications, which form a major part of the technical documentation in a BDD project.

You've seen how high-level acceptance tests describe and specify the behavior of the application as a whole. You've also seen how BDD unit tests describe and specify

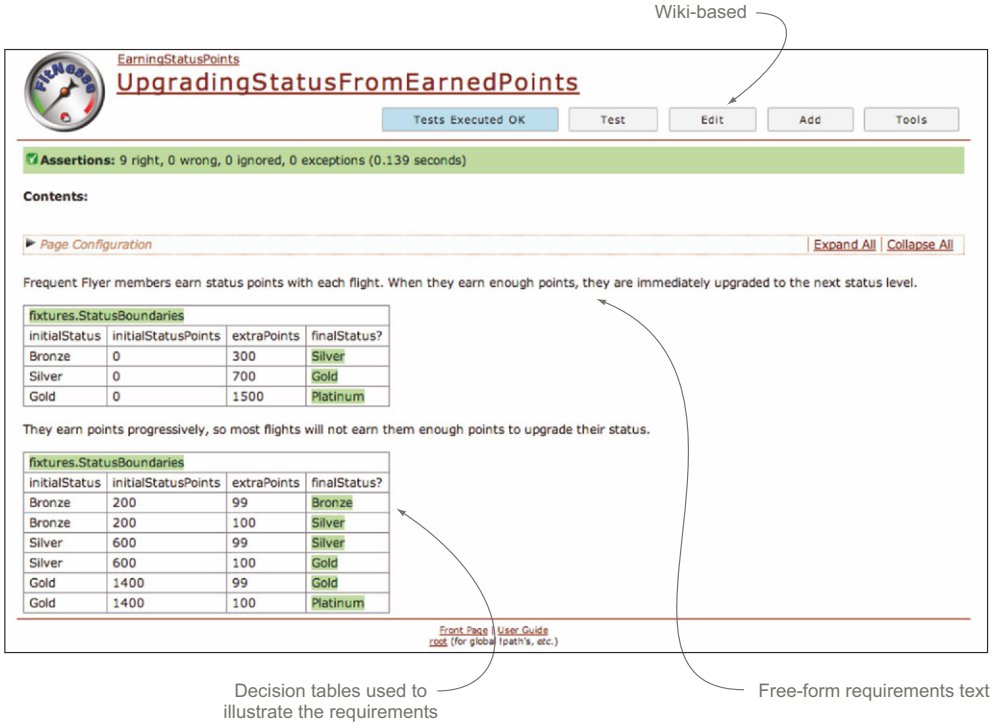


Figure 11.11 In FitNesse, you document requirements in a wiki.